

Movies Database Schema Design Document.docx

Movies Database System Schema Design Document

1. Overview

This document defines the relational schema for a movie ratings system with three core entities: MOVIES, USERS, and RATINGS. It describes the table structures, primary and foreign keys, relationships, and implementation considerations needed to support movie metadata, user profiles, and rating activity.

2. Entity Definitions

2.1 MOVIES

The MOVIES table stores metadata about each movie.

Primary Key:

- MovieID

Attributes:

- Title
- Year
- Genres
- Language
- Country
- TotalViews

2.2 USERS

The USERS table stores profile information and engagement metrics for each user.

Primary Key:

- UserID

Attributes:

- Age
- Gender
- Country
- SubscriptionStatus
- TotalWatchTime
- Device

2.3 RATINGS

The RATINGS table stores individual rating events and links users to movies.

Primary Key:

- RatingID

Foreign Keys:

- UserID → USERS.UserID
- MovieID → MOVIES.MovieID

Attributes:

- Rating
- Timestamp

3. Relationships

The schema uses a one-to-many relationship model:

- One movie can have many ratings
- One user can submit many ratings
- Each rating belongs to exactly one user and one movie

This is implemented through the foreign keys in the RATINGS table:

- RATINGS.MovieID references MOVIES.MovieID
- RATINGS.UserID references USERS.UserID

4. Constraints and Rules

The following constraints and rules should be applied to maintain data integrity:

- Enforce referential integrity on both foreign keys in RATINGS
- Define delete and update behavior according to policy, such as CASCADE or RESTRICT
- Validate field data types, including Rating as a numeric type and Timestamp as a datetime type
- Add indexes on RATINGS.MovieID and RATINGS.UserID to improve join performance and lookups

5. Implementation Notes

When implementing the schema:

1. Create the MOVIES, USERS, and RATINGS tables with the fields listed above.
2. Apply primary key and foreign key constraints.
3. Insert sample records into MOVIES and USERS.
4. Insert related records into RATINGS.
5. Test the relationships using join queries to confirm the schema behaves as expected.

6. Validation

Before finalizing the schema, verify that:

- All keys are correctly defined
- All relationships resolve properly

- Data types match the intended business rules
- Indexes are in place for performance
- Referential actions align with the database policy